

Subroutines in Machine Language for Microsoft BASIC Interpreter for MS-DOS

Martin Hepperle, 2024, 2026

After Microsoft had developed their BASIC interpreter for many CP-M systems with 8-bit CPUs and 64 kB memory, the system was also transferred to Intel processors with 16-bit registers and MS-DOS. These versions maintained the 8-bit BASIC structure. By using a common, 8-bit based code-base and the segmented architecture of the new Intel processors, Microsoft could rapidly provide a BASIC for these new systems. The capabilities of the new computers to access up to 1 MB of memory were not yet fully used. Only a separation of the BASIC system into code and data areas was implemented – each area could be 64 kB. The remaining memory of the computer stayed unused (it could, cumbersome and error prone, accessed byte-by-byte using `DEF SEG` and `PEEK/POKE` statements).

Some functions are difficult or impossible to implement in BASIC. Sometimes the signed integer number representation in BASIC can be a hindrance. In such cases it can be provide the desired routine in machine language. However, calling a subroutine in machine language from BASIC always involves some overhead. Therefore, it is often faster to perform small tasks directly in BASIC – the multiplication of an integer number with the constant 2 executes faster in BASIC than calling a subroutine in machine language which performs this operation with a faster CPU operation of a left shift by one bit. Calling the machine language routine requires saving registers, setting up parameter and the return address on the stack and when returning to BASIC these registers must be restored. All this costs time. If not sure, one has to try which variant is faster.

There are several ways to manage subroutines in machine language together with the BASIC interpreter BASIC Interpreter in the memory of the computer.

Here we demonstrate two methods how to load and execute subroutines in machine language from BASIC.

Method A: Routines inside the BASIC data segment

In this variant the byte sequence comprising the machine language routine are converted by `READ` statements from `DATA` statements into integers and then stored by `POKE` commands inside an array variable (usually declared as `INTEGER`). For this purpose the starting address of the array is obtained with `VARPTR` and then each byte is copied to the following memory cells.

The routine at this starting address can later be called by the `USR` or `CALL` statements. An advantage of this method is that the machine code resides completely inside the BASIC variables and no external files are required. A disadvantage is that the cryptic `DATA` statements must be entered by hand (cumbersome and error prone) or have to be created by an external program and then must be integrated into the BASIC. Also, the `DATA` statements consume valuable memory in the BASIC program text in addition to the bytes in the `INTEGER` array.

Instead of reading `DATA` statements and storing the bytes with `POKE` commands it is also possible to read the code bytes quickly with the `BLOAD` command from a file into the `INTEGER` array. This leaves a shorter BASIC program and it is not necessary to change the BASIC program if the machine language

routine is changed. The conversion of the binary program into the specific **BLOAD** format can be performed with a small utility program [1], or simply be considered when writing the assembler code. On the other hand the binary must be provided as an extra file at runtime.

A third variant excludes an area at the upper end of the BASIC data segment from the usage by the BASIC system. For this purpose the command line option **/M** is used to lower the upper end address in BASIC's data segment. The binary code can now be loaded from a file into this area with the **BLOAD** command and no array variable is required. Again the binary file must be present at runtime.

A drawback of these three variants of method „A“ is that the machine code consumes memory inside the BASIC data segment, leaving less room for BASIC variables. An annoyance is the fact that BASIC moves the location of array variables in memory whenever a new simple variable (numeric or string) is created. Therefore it is necessary to determine the address of the machine code routine in the **INTEGER** array by using the **VARPTR** keyword only after all simple variables have been allocated, i.e. when at least once a value has been assigned to each of them. This requires initializing all (really all!) simple variables at the start of the program. Another safe (but slower) approach is to determine the address every time just before a **CALL** or **USR** statement is used.

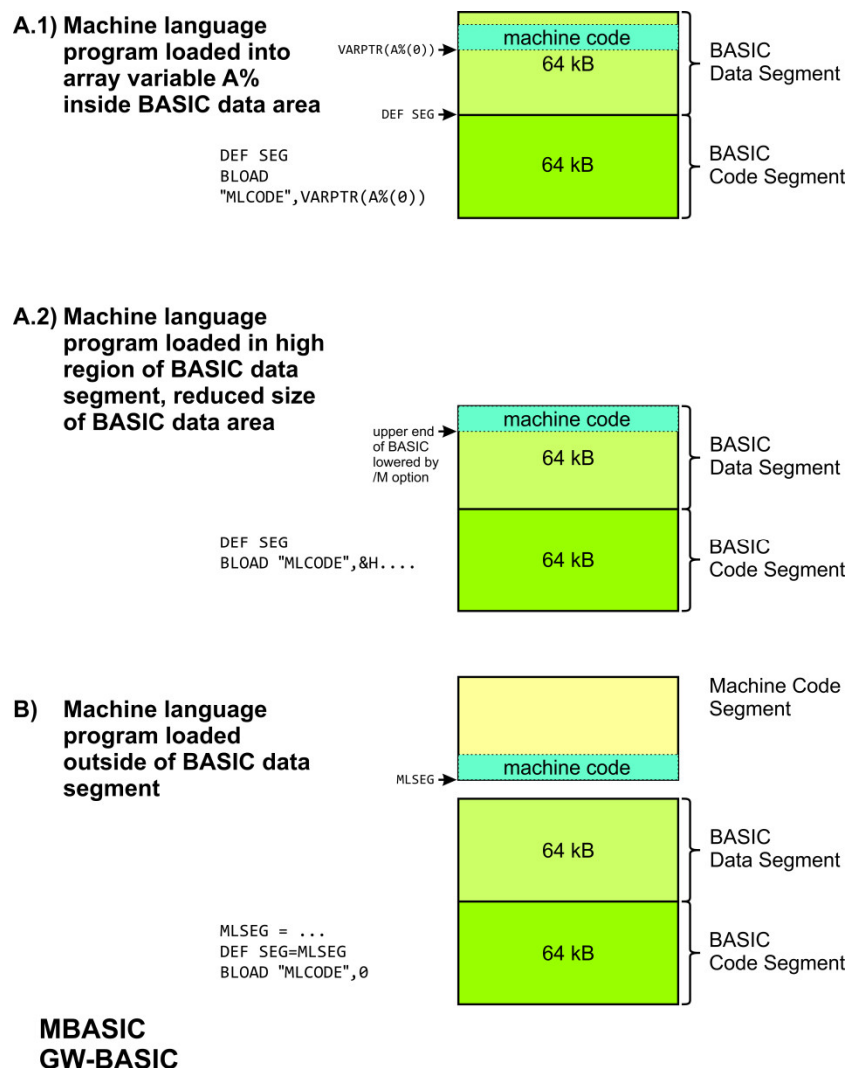


Figure 1: Options to organize routines in machine language in memory. A) Inside and B) outside of the BASIC memory area.

Method B: Routines above the BASIC data segment

In order to support the segmented memory architecture of the 16-bit Intel processors, the MS-DOS versions of BASIC introduced the new keyword `DEF SEG`. This allows defining a segment address for the subsequent execution of the functions and statements `PEEK`, `POKE`, `CALL`, `CALLS`, `USR`, `BLOAD`, `BSAVE`. This segment can also be located outside of the BASIC data area. Additionally, the keyword `CALLS` was added which transfers parameters as a *Segment:Offset* (FAR) address instead of the der 16-bit *Offset* (NEAR) address used by the `CALL` statement.

With the help of `DEF SEG` it is possible to use subroutines in machine language which are located outside of the limited BASIC data area.

Of course, one has to know their location and the storage area must not interfere with the BASIC system or the MS-DOS operating system. Most MS-DOS systems provide enough free memory above the two 64-kB segments used by BASIC.

An advantage of this method is that this area is not affected by BASIC and no movement of fuction addresses occurs. Thus the `VARPTR` function can be used once at the start of the BAIC program without the risk of invalid pointers when they are used with a `CALL` or `CALLS` statement.

There is a risk with very small system of 256 kB RAM that device drivers may use the upper area of ram. This could be checked, but the probability is so small that we ignore it.

The first step is to find which segments are used by BASIC – depending on system configuration this can vary.

Surveying the undocumented internal BASIC data structures we find a variable which stores the segment address of the BASIC data segment. This variable is set or reset by the `DEF SEG` statement. The location of this variable differs in the widely used BASIC versions MBASIC 5.28 and GWBASIC 3.32 but it can be used in the same way.

Alternatively one can also use a small machine language program to find out the currently used data segment.

Considering the maximum size of the data segment of 64 kB allows calculating the address of the free memory above the BASIC system.

In order to avoid the lengthy `POKE` sequences, we use the `BLOAD` command for loading the binary file.

We also use the `CALL` statement because it allows for an arbitrary number of parameters. For return values it makes sense to add one or more parameters. The `USR` function allows only for passing a single parameter. The only advantage of `USR` is that it actas ass a function and returns a value, which can lead to more structured programs and `USR` is also available in much older Microsoft BASIC versions.

A binary subroutine module is created by these steps:

- Write the `.ASM` assembler source code which can consist of one or more subroutines. The load address is defined with `ORG 0H`.
- Assemble the source code to an `.OBJ` file. I use Microsoft `MASM` for this purpose.
- Link the `.OBJ` file into a `.BIN` file, which can be performed directly with Microsoft `LINK 5.15`. Alternatively older versions of `LINK` can be used; these create an `.EXE` file, die man which can then be converted with `EXE2BIN` to create a `BIN` file.
- The sequence `MASM` and `LINK` and optionally `EXE2BIN` can be wrapped into a `BATCH` script.

The following steps can be used to write the BASIC program:

- At the start of the BASIC program we determine a high segment address above the BASIC segment and then we load the .BIN file with BLOAD die into this space. This load has to be performed only once when the program starts. This segment address will be required later again.

In order to obtain an address above the BASIC data segment we can read the internal BASIC system variable SAVSEG and add &H1001 (each increment of a segment address increases the physical address by 16 Bytes. Therefore, this increments adds $16 \times \&H1000 = 65535$ bytes plus a safety buffer of one paragraph of 16 bytes).

The location of the variable SAVSEG depends on the BASIC version:

```
330 REM =====
340 REM --- MSBASIC 5.28 for MS-DOS (internal BASIC variable SAVSEG at &H4D0)
350 PRINT "For MS BASIC 5.28" : A=&H4D0 : GOTO 400
360 REM =====
370 REM --- GWBASIC 3.23 for MS-DOS (internal BASIC variable SAVSEG at &H496)
380 PRINT "For GW BASIC 3.23" : A=&H496 : GOTO 400
390 REM
400 DEF SEG : REM default segment allows reading internal BASIC variable
410 MLSEG=PEEK(A)+256*PEEK(A+1)+&H1001
420 . . .
```

- Next we define for each subroutine a variable with the offset into this high segment (function pointer). If more than one subroutine is contained in a single binary, we can include a jump table at the start of the code. This allows defining these offsets simply by a systematic increment according to the length of the jump instruction of 3 bytes. The small disadvantage of introducing another jump is balanced by a clear structure and a less error prone program. Many assemblers optimize the 3-byte NEAR JMP instruction into a 2-byte SHORT JMP, but insert a NOP command so that the distance of 3 bytes between the JMP commands is maintained. The definition of these function pointers has to be performed only once at the start of the program.

In the example BASIC program this looks like so:

```
460 DEF SEG=MLSEG
470 BLOAD "BITS.BIN",0
480 REM ----- function pointers = position in der jump table
490 LSHIFT=0
500 RSHIFT=3
```

The head of the corresponding assembler module with several functions looks like this:

```
; -----
ENTRY:
;   jump table, if more than one function is implemented
;   in this module
;   JMP NEAR PTR LSHIFT ; Offset = 0  each NEAR JMP uses 3 Bytes
;   JMP NEAR PTR RSHIFT ; Offset = 3
```

- Before calling sub a subroutine, one has to select the segment address for the following CALL by using the DEF SEG = statement. All parameters must be variables because they are passed as pointers. It is not possible to use constants like in CALL SUB(1), instead one has to write something like I%=1 : CALL SUB (I%).

A typical calling sequence for the first function LSHIFT is shown below:

```
640 REM select code-segment for CALL
650 DEF SEG=MLSEG
660 A%=16383
670 PRINT A%;" << 1 = ";
680 CALL LSHIFT(A%)
690 PRINT A%
```

Additionally, the following points should be taken into account:

- Intel Processors use a *decrementing* stack. If a value is **PUSHed** to the stack the processor first decrements the value of the stack pointer **SP** by the size of the value to be pushed (e.g. 2 bytes for a 16-bit offset) and afterwards copies the value to this new address. The reverse operation **POP** first copies the value from the address in **SP** and then increments the stack pointer **SP** accordingly. **SP** always points to the last value pushed to the stack.
- All subroutines are called with a **FAR CALL**. Therefore they must be declared as **PROC FAR** in the assembler code so that the **RET** instruction leaves the stack in a proper state.
- Parameters are transferred as 16-bit offsets into the BASIC data segment. The **CALL** statement pushes them in left-to-right order to the stack, followed by the 16-bit program segment **CS** and the 16-bit program counter **IP** for the **FAR** return. When the subroutine is entered, the stack looks like so:

```

stack
| ... high addresses
|   +6  A% offset address 1. parameter
|   +4  B% offset address 2. parameter
|   +2  return address segment CS
| SP -> +0  return address offset IP
| ... low addresses

```

- When returning to BASIC the binary program must clean up the stack. If for example two parameters have been transferred, the routine must execute a **RET 4** (this takes the 2×2 Bytes for the parameter-addresses off the stack before executing a **RET FAR** to the address **CS:IP** which is still on the stack). If instead of **CALL** the BASIC command **CALLS** had been used, the parameter addresses had been transferred as **FAR** addresses and we would have to use a **RET 8**.
- The registers **DS**, **ES**, **SS** always contain the address of the BASIC data segment. **CS** contains the segment address of the binary routine. The subroutine must preserve the registers **BP**, **DS**, **ES**, **SS** and has to restore them before returning.
- Numeric variables are transferred as addresses to their BASIC number representation. For example in case of an integer variable:

```
Integer parameter: Address -> 2 bytes Integer value
```

- Arrays can be transferred by specifying the first element as the parameter. The subroutine receives the address of this first element and all following elements follow in memory. In case of multi-dimensional arrays, the first index varies first. Floating point numbers use the „Microsoft Binary Format“ (MBF) and not the IEEE-754 Standard (this standard was used later starting with QuickBasic 4.5).
- When strings are transferred to the subroutine the address of a “string descriptor” structure is passed. It consists of a single byte for the current length of the string, followed by a 16-bit address of the actual string content.

```
string parameter: address -> 1 byte string length (max. 255)
                    2 Bytes address of the string
```

Strings can be read and written, but it is not allowed to change their length. This means, that the calling program must pre-allocate the string to the desired length, if the subroutine shall return something in a string. Therefore the example functions **BINSTR** and **HEXSTR** first test the string length to make sure there is enough space.

Note that strings which are statically defined directly in the BASIC program are stored in the program text, not in string space.

```
100 A$="Hello World"
110 CALL UPC$(A$)
```

When the program is executing the `UPC$` function, it modifies the program text and a subsequent `LIST 100` displays this line:

```
100 A$="HELLO WORLD"
```

The subroutine has changed the program text.

If this behavior is not desired, the string has to be allocated dynamically in the string space, for example by concatenation of two strings:

```
100 A$="Hello"+" World"  
110 CALL UPC$(A$)
```

A subsequent `LIST 100` displays the same as the initial line:

```
100 A$="Hello"+" World"
```

Example Routines BITS for MSBASIC 5.28 and GWBASIC 2.32

The following subroutines are contained in the example files `BITS1.ASM` and `BITS2.ASM`. They are accompanied by the test programs `BITS1.BAS` and `BITS2.BAS`. They demonstrate two options for integrating machine language routines in BASIC programs.

```
CALL LSHIFT(A%)
```

Shifts the 16-bit value `A%` by 1 bit left. The rightmost bit 0 will be zeroed.

```
CALL RSHIFT(A%)
```

Shifts the 16-bit vert `A%` by 1 bit to the right. In contrast to the BASIC operator `\` the sign bit 15 is not copied but zeroed.

```
CALL BITAND(A%,B%)
```

Bitwise And. Sets the 16-bit value of `A%` to `(A% AND B%)`. The same effect can be obtained by the built-in keyword `AND`.

```
CALL BITIOR(A%,B%)
```

Bitwise Inclusive Or. Sets the 16-bit value `A%` to `(A% OR B%)`. The same effect can be obtained by the built-in keyword `OR`.

```
CALL BITEOR(A%,B%)
```

Bitwise Exclusive Or. Sets the 16-bit value `A%` to `(A% XOR B%)`. The same effect can be obtained by the built-in keyword `XOR`.

```
CALL LROT(A%)
```

Rotates the bits in the 16-bit value `A%` by 1 bit to the left. The leftmost bit 15 is rotated into bit 0.

```
CALL RROT(A%)
```

Rotates the bits in the 16-bit value **A%** by 1 bit to the right. The rightmost bit 0 is rotated into bit 15.

CALL FLIP(A%)

Inverts the bits in the 16-bit value **A%**. '1' bits change to '0' and vice versa. The same effect can be achieved by the BASIC keyword **NOT**.

CALL GETSEG(CS1%,CS2%,DS%,ES%,SS%)

Returns code, data and extra segment registers.

The first code segment parameter **CS1%** is the BASIC Code Segment, **CS2%** is the segment used by the machine language routine (has been set before by the **DEF SEG** statement).

CALL PEEK16(SEG%,OFF%,VAL%)

Returns in **VAL%** the 16-bit word at **SEG%:OFF%**. Replaces two **PEEKs** and enclosing **DEF SEGs**.

CALL POKE16(SEG%,OFF%,VAL%)

Copies the 16-bit word **%VAL** to the memory location **SEG%:OFF%**. Replaces two **POKE** statements with enclosing **DEF SEGs**.

CALL UPC(STR\$)

Converts the characters in the string **STR\$** into capital letters [a-zäöü] → [A-ZÄÖÜ] (ASCII and German umlauts).

CALL LWC(STR\$)

Converts the characters in the string **STR\$** into lower case letters [A-ZÄÖÜ]→[a-zäöü] (ASCII and German umlauts).

CALL BINSTR(NUM%,STR\$)

Converts the unsigned integer **NUM%** into a string **STR\$** of binary digits (0, 1).

STR\$ must be pre-allocated with arbitrary characters to a length of 8 or 16 characters for converting a byte respectively a 16-bit word.

CALL HEXSTR(NUM%,STR\$)

Converts the unsigned integer **NUM%** into a string **STR\$** of hexadecimal digits (0-9, A-F).

STR\$ must be pre-allocated with 2 or 4 characters for converting a byte respectively a 16-bit word.

CALL REVSTR(STR\$)

Reverses the character sequence in the string **STR\$**.

Sources

[1] C. A. Crayne, D. Girard, „The Serious Assembler“, Baen, 1985.